



Centre for Social Conflict and Cohesion Studies

Taller de R

Módulo 2: Gestión de Datos

Benjamín Muñoz

6 de Enero

Escuela de Verano de Métodos Mixtos 2017

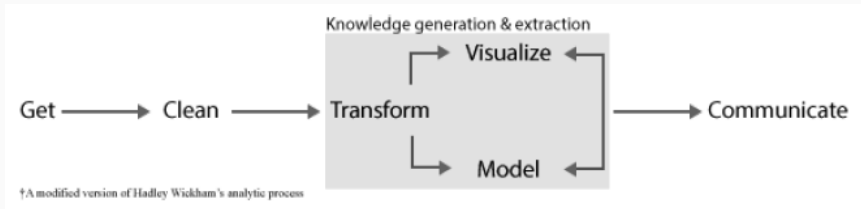
Tabla de Contenidos

1. Introducción
2. Importación desde archivos planos
3. Funciones Básicas de Manejo de Datos I
4. Importación desde Excel
5. Importación desde otros softwares (SPSS y Stata)
6. Funciones Básicas de Manejo de Datos II

Introducción

- R es un software orientado a objetos: todo es un objeto en el espacio de trabajo, desde un número a una base de datos que tenga todas las cuentas activas de Facebook (1.79 billones).
- Ya vimos que existen distintas estructuras de datos, sin embargo lo más frecuente es querer trabajar con bases de datos: arreglos rectangulares de datos con filas que representan observaciones y columnas representando variables.
- Este es el concepto guía de la sesión. Nos centraremos en ver como obtener y manipular bases de datos.

Figura 1: Flujo de Trabajo Empírico



Data Wrangling

- Tomar fuentes de datos brutas y sucias y transformarlas en algo útil para ser analizada.
- Involucra importar datos, limpiar la información y realizar las transformaciones pertinentes para una determinada aplicación. Es normal que implique una buena parte del trabajo de código.



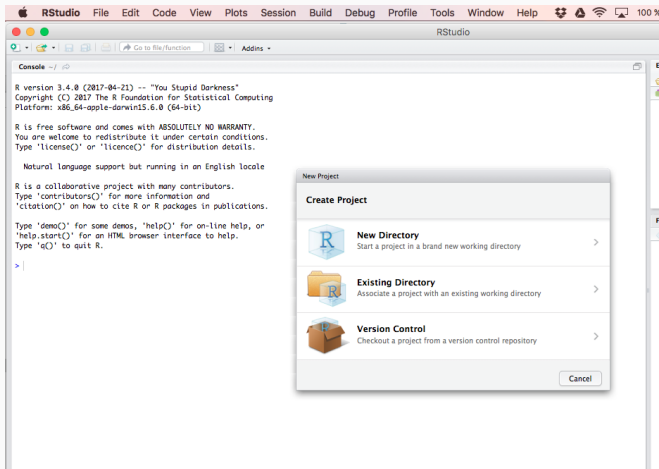
Figura 2: Procesamiento de Datos

- R tiene su propio formato de datos, cuya extensión es `.RData`. Más adelante veremos cómo generar dichos archivos.
- En el espacio de trabajo se encuentran los objetos. De todos modos, R también almacena el historial de comandos utilizados.
- El directorio de trabajo es la localización dónde se almacenan los archivos que utiliza R. Los comandos `getwd()` y `setwd()` son cruciales para su determinación.
- No obstante lo ideal es trabajar con directorios relativos, en la práctica usaremos directorios absolutos.

Opciones del directorio de Trabajo

- Al fijar un directorio de trabajo, R sabe dónde realizar búsquedas de archivos y también dónde almacenarlos (siempre que no se especifique explícitamente otro directorio).
- El problema es que los directorios de trabajo son dependientes del computador utilizado y de la posición de éstos. Un cambio en la localización (por ejemplo, trasladar la carpeta) exige corregir el código. Tampoco facilita el trabajo colaborativo (cada usuario deberá fijar el directorio de trabajo).
- La alternativa es el uso de proyectos (archivos .Rproj, disponibles en RStudio) para contener los archivos. Al trabajar con proyectos se superan los problemas anteriores.

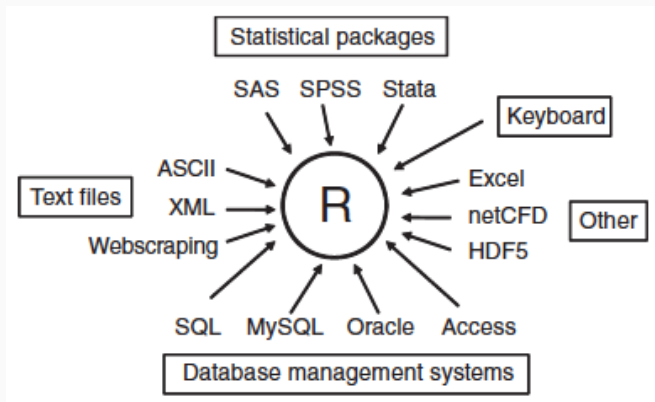
Figura 3: Crear Proyectos en RStudio: File/New Project



- Se generan únicamente en RStudio por Menú. En File elegir New Project.
- Las opciones básicas son dos: crear un nuevo directorio o usar uno existente (se refieren a una carpeta). Al elegir cada opción se puede navegar para fijar su locación.
- Posteriormente se genera un archivo .Rproj que debe ser cargado al comienzo de cada sesión de trabajo. Con esto R sabrá dónde buscar los archivos de manera fácil.

- La forma más sencilla de obtener un conjunto de datos es activarlos por medio de un paquete previamente instalado.
- Una segunda opción es cargar un objeto en formato `.RData`. Para esto usaremos el comando `load( )`.

Figura 4: Importación de Datos a R



Importación desde archivos planos

- La función `read.table( )` es un comando multipropósito incorporado a la versión base de R. La utilizaremos para importar distintos tipos de archivos planos a R.
- También existen las funciones `read.csv` y `read.delim`, pero son casos especiales de `read.table` en que se ajustan los valores por defecto para volver más eficiente la importación.

read.table(file,header=F,sep=)

- El comando tiene 22 parámetros, pero los más importantes son:
 - `file`: nombre del archivo a importar.
 - `header`: T si primera fila son nombres de variables, F si son valores. En dicho caso se pueden agregar `col.names`.
 - `sep`: indica el separador de los valores en el archivo.
 - `dec`: indica si los decimales son separados por puntos o comaas.
- Para revisar los restantes ingresen `?read.table()`

Funciones Básicas de Manejo de Datos I

Comandos para Revisión de Datos

- Siempre es bueno darle una mirada sencilla a los datos:
- La alternativa más sencilla es llamar al objeto de la base de datos.
- Existen alternativas:
 - `head(base_de_datos)`
 - `tail(base_de_datos)`
 - `summary(base_de_datos)`
 - `View(base_de_datos)`
 - `glimpse(base_de_datos)`
 - `base_de_datos <- tbl_df(base_de_datos)`
- Probemos estos comandos:

Importación desde Excel

- Es probablemente el formato más complejo para importar datos. En la medida que sea posible **eviten** usar archivos de Excel (.xls o .xlsx).
- Esto se debe a distintos aspectos. Los básicos son el formato múltiple (Libro/Hojas), sus dimensiones (inicio y término de la hoja) y estructura.
- Se han diseñado múltiples paquetes para esta tarea. Sin embargo, varios presentan *bugs* o dependen de algunos elementos (versiones de Java).
- Usaremos el paquete `readxl` y su función `read_excel`. Es bastante sencilla, únicamente necesita especificar el directorio en que se encuentra el archivo (usando “ ”).

`read_excel(path=, ...)`

- `sheet= 1` Cuál es la hoja del libro a importar.
- `colnames=TRUE` Si la primera fila son nombres de variables.
- `col_types=NULL` el comando determina la clase de variables
- `na=""` Que valores se convierten en NA
- `skip=0` Número de filas que se salta antes de lectura.

Importación desde otros softwares (SPSS y Stata)

- Los softwares estadísticos comerciales más usados: Stata, SPSS y SAS tienen formatos propios para almacenar datos.
- La importación se realiza por medio de funciones específicas en paquetes. Utilizaremos `foreign` y `sjmisc`.

Comandos en sjmisc

- `read_spss(path, atomic.to.fac = FALSE, tag.na = FALSE)`
- `read_sas(path, path.cat = NULL, atomic.to.fac = FALSE, enc = NULL)`
- `read_stata(path, atomic.to.fac = FALSE, enc = NULL)`
- `path` corresponde al directorio.
- `atomic.to.fac` convierte vectores atómicos en factores.
- `enc` alude al encoding/lenguaje de códigos. El más frecuente es UTF 8.

Comandos en foreign

- `read.spss(file, use.value.labels = TRUE, to.data.frame = FALSE ...)`
- `read.dta(file, convert.dates = TRUE, convert.factors = TRUE, missing.type = FALSE, ...)`
- El `file` se especifica como un nombre en el directorio de trabajo (entre comillas) o con el argumento `file.choose()`.
- El principal defecto de estas funciones es que son antiguas (no compatible con datos Stata 13 y 14).
- Hay otros argumentos adicionales que les añaden flexibilidad. No facilitan el uso de etiquetas.

- Poco usadas en R ya que se pueden crear objetos ad hoc o ajustar los atributos en su reemplazo.
- Las etiquetas quedan como atributos pero son de difícil manipulación. Alternativas son usar como etiqueta de variable el nombre de ésta (y llamarla en base a su posición) y de etiquetas de valores fijarlas con `factor( )`.

- La ventaja de utilizar el paquete `sjmisc` es que importa, de manera sencilla las etiquetas provenientes de bases de datos en formatos SPSS (`.sav` y `.por`) y STATA (`.dta`).
- De este modo, las etiquetas de valores y variables quedan almacenados en el marco de datos. Sin embargo, su uso no es sencillo. Se pueden usar como argumentos en la generación de gráficos. La familia de paquetes `sjstats` y `sjPlot` tiene funciones que utilizan de manera automática las etiquetas, especialmente con el fin de permitir visualizaciones de mejor calidad.

- Existen múltiples modos de descargar datos desde internet.
- Ya vimos algunas funciones básicas. Otra alternativa más genérica es el web-scraping.
- En este curso nos centramos en datos rectangulares, por lo que no abarcaremos este contenido. En cambio, conoceremos el uso de enlaces locales para la extracción de datos.

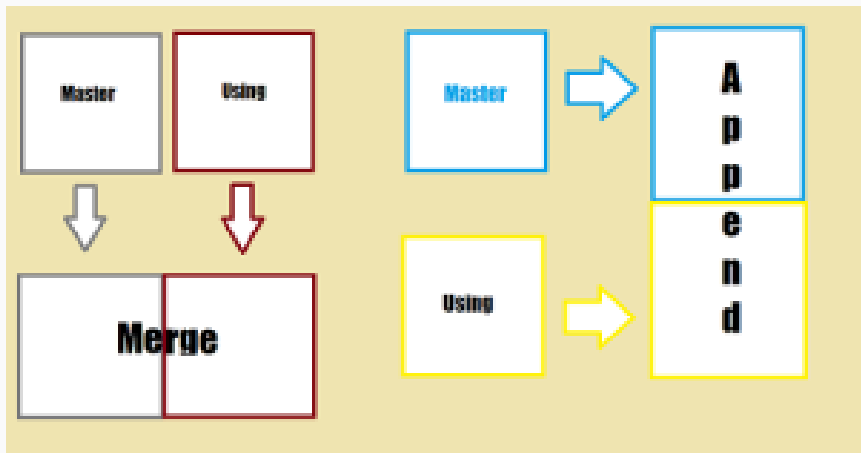
Funciones Básicas de Manejo de Datos II

Otros Comandos para Revisión de Datos

- Vimos comandos genéricos de revisión de datos. Hay algunos más específicos:
 - `str(base_de_datos)`: enlista la estructura (nombres, tipos y algunos valores).
 - `class(base_de_datos)`: clase del objeto
 - `mode(base_de_datos)`: modo en que es almacenado
 - `names(base_de_datos)`: enlista los nombres de los elementos contenidos.
- Revisemos su uso en el código. Decidir cómo manipular datos siempre requiere inspeccionar los objetos.

- Suma horizontal: agregar variables `append`
- Suma vertical: agregar observaciones `merge`
- Cambiar unidad de observación. `aggregate`
- Cambiar dimensiones de los datos. `subset`, `melt`, `cast`
- Recodificación. `recode`

Figura 5: Combinar Bases de Datos



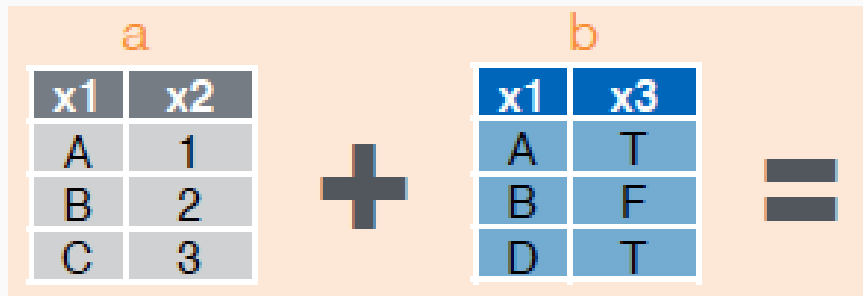
- Existen dos operaciones básicas para combinar bases de datos:
 1. **Suma horizontal** o agregar variables (información sobre los casos).
 2. **Suma vertical** o agregar casos
- La preocupación fundamental es la correspondencia de la información al momento de combinar los elementos. El segundo aspecto a considerar son las dimensiones de los objetos a combinar.
- El comando básico para agregar casos es `rbind()` (significa row bind o unir por filas). El criterio fundamental es que los nombres de las variables sean idénticos. Es decir `rbind` sólo funciona si están, en ambas bases de datos, las mismas variables.

- La función más sencilla para agregar columnas es `cbind()` (column bind). Sin embargo, **no es recomendable su uso**, ya que ignora el orden de los casos. Es preferible usar como función básica `merge()` el cual permite indicar cuáles son las variables identificadoras de casos.

¿Cómo usar la función `merge`?

- `nueva_base <- merge(x=base01, y=base02, by= , all = )`
- Se indican las bases a combinar (sólo pueden ser dos).
- El criterio `by= "variable"` permite indicar cuál es la(s) variable(s) identificadora(s) de casos. Aquí se asume que tiene el mismo nombre en ambas bases. En caso contrario, utilizar `by.x= "varX"` y `by.y= "varY"`, señalando el nombre en cada objeto.
- Con el argumento `all= ...` se puede indicar si se preservan todas las observaciones, las presentes en ambas bases o sólo las provenientes de una de las dos bases.

En el paquete dplyr hay múltiples funciones de combinación



Opciones Posibles: Agregar casos y variables simultáneamente

x1	x2	x3
A	1	T
B	2	F
C	3	NA

dplyr::left_join(a, b, by = "x1")
Join matching rows from b to a.

x1	x3	x2
A	T	1
B	F	2
D	T	NA

dplyr::right_join(a, b, by = "x1")
Join matching rows from a to b.

x1	x2	x3
A	1	T
B	2	F

dplyr::inner_join(a, b, by = "x1")
Join data. Retain only rows in both sets.

x1	x2	x3
A	1	T
B	2	F
C	3	NA
D	NA	T

dplyr::full_join(a, b, by = "x1")
Join data. Retain all values, all rows.

- Por defecto las funciones están diseñadas para hacer simultáneamente la agregación de casos y variables. Funcionan igualmente si la operación sólo implica uno de los dos aspectos.
- Se pueden especificar múltiples variables identificadoras de casos por medio `c()`. De todos modos, opera sobre dos bases de datos.
- También existen funciones de combinación más simples en `dplyr`:
 - `bind_rows`: permite combinar muchos elementos y empareja según los nombres de las variables (si no coinciden, genera columnas adicionales e imputa valores perdidos).
 - `bind_cols`: permite combinar muchos objetos agregando columnas. Lo hace según su posición en los objetos (todos deben tener el mismo número de filas).

Redimensionamiento de base de datos

- Dentro de este paraguas existen múltiples posibilidades. Nos centraremos en dos operaciones: extraer un subconjunto de datos y el cambio de formato (wide/long).
- Deben tenerse en cuenta que hay otras opciones posibles: transponer bases de datos, combinar múltiples columnas en una, separar una columna en varias, etc.

- Para los usuarios de Stata, comando como `keep` y `drop` les deben resultar familiares.
- Sin embargo, en R hay una estrategia más sencilla: el comando `subset`.

¿Cómo usar la función `subset`?

- `nueva_base <- subset(base_original, subset, select)`
- Para generar la nueva base de datos hay dos argumentos, `subset` y `select`.
- El primero corresponde a las filas a preservar y se explicita por medio de operaciones lógicas.
- El segundo permite elegir columnas, lo cual se hace por medio de su nombre.

Uso de comando subset

- `> data("mpg")`
- `> mpg`
- `# A tibble: 234 x 11`

	manufacturer	model	displ	year	cyl	trans
	< chr >	< chr >	< dbl >	< int >	< int >	< chr >
1	audi	a4	1.8	1999	4	auto(l5)
2	audi	a4	1.8	1999	4	manual(m5)
3	audi	a4	2.0	2008	4	manual(m6)
4	audi	a4	2.0	2008	4	auto(av)

- `# ... with 230 more rows`

Uso de comando subset

- Revisemos algunas operaciones comunes con el ejemplo anterior.

Según un valor numérico

- `mpg_sub <- subset(mpg , subset = displ > 2,4)`
- `mpg_sub <- subset(mpg , year == 2008)`
- `mpg_sub <- subset(mpg , cyl != 5 )`

Según los valores de un caracter

- `mpg_sub <- subset(mpg , manufacturer == 'audi')`
- `mpg_sub <- subset(mpg , manufacturer == 'audi' | manufacturer == 'jeep')`
- `mpg_sub <- subset(mpg , manufacturer == 'dodge' & class == 'pickup')`

Uso de comando subset

- También se pueden incorporar funciones dentro de subset.
- El comando `na.omit` realiza listwise deletion sobre la base de datos, eliminando todas las observaciones que contengan uno o más valores perdidos (en cualquier variable).
- Esto no siempre es deseable, ya que es preferible señalar cuáles variables guían la eliminación de casos. Para esto usamos `is.na` dentro de subset.

Algunos casos especiales

- `efc_sub <- subset(efc , is.na(quo1_5)==F)`
- `efc_sub <- na.omit(efc)`

Cambio de Unidad de Observación de Base de Datos

- La operación de agregación puede hacerse de modo de lograr un único valor resumen de la base de datos. Sin embargo, lo más interesante es su aplicación a grupos de casos, lo que corresponde a cambiar la unidad de observación.



Cambio unidad de Observación

- Corresponde al proceso en que se redimensiona una base de datos, de modo de agregar los casos a una unidad de análisis superior (por ejemplo, pasar de individuos a familias, de comunas a países, etc).
- Para que funcione de manera adecuada, se necesita de una variable llave que tome el mismo valor en todos los casos que se desean agregar. Chequear dicha variable(s) es fundamental.
- Revisaremos dos comandos: aggregate y summarize.

Uso de comando aggregate

- Tengo una base de datos a nivel de comunas con la votación de la derecha y concertación en elecciones de diputados. Deseo agregarla a nivel regional (deseo sumar los votos y mi variable llave es region.
- ```
base_reg <- aggregate(cbind(Alianza_v, Concertacion_v, Votacion_comunal) ~ region, data= elecciones_comunal, sum)
```

## aggregate

- `aggregate(x, by, FUN, ...)`
- `x` se refiere al objeto a agregar, `by` a la variable llave (`region`, la especificamos con el símbolo `~` ) y `FUN` a la función que se usa para agregar. Otras opciones son `mean`, `median`, `var`, etc.

## En dplyr existe summarise y group.by

- `mpg_group <- group_by(mpg, manufacturer) # No cambia dimensiones`
- `mpg_agg <- summarise(mpg_group, displ_mean=mean(displ), displ_sd=sd(displ), n())`
- Con el comando anterior, calculamos la media, desviación estándar (para la variable displ) y número de observaciones por grupo.
- La ventaja de summarise es su flexibilidad y el amplio rango de funciones que pueden usarse en el cambio de la unidad de análisis (algunas de las cuáles son útiles para trabajar con variables de carácter).
- Para mayores detalles revisen la Cheat Sheet de Data Wrangling (RStudio).